



JavaScript中级-DOM

JSDOM

001.DOM简介：文档对象模型

- JavaScript 可以改变页面中的所有 HTML 元素
- JavaScript 可以改变页面中的所有 HTML 属性
- JavaScript 可以改变页面中的所有 CSS 样式
- JavaScript 可以删除现有的 HTML 元素和属性
- JavaScript 可以添加新的 HTML 元素和属性
- JavaScript 可以对页面中所有现有的 HTML 事件做出反应
- JavaScript 可以在页面中创建新的 HTML 事件

HTMLDOM：是HTML标准对象模型和编程接口

- HTML 元素作为对象
- 所有 HTML 元素 的属性
- 访问所有 HTML 元素的方法
- 所有HTML 元素的事件

002.DOM方法：

DOM方法指可以在HTML元素上执行的操作

DOM属性值可以设置或更改HTML的元素的值

DOM编程接口：在 DOM 中，所有 HTML 元素都被定义为**对象**。编程接口是每个对象的属性和方法。可以使用 JavaScript（以及其他编程语言）访问 HTML DOM。

getElementById方法与innerHTML属性，最常用于获取或替换HTML元素的内容

003.DOM文档：文档对象是网页中所有其他对象的所有者，理解问HTML页面

想要访问HTML中的元素，需要先访问文档对象，即document

查找HTML对象：

document.getElementById (id)，通过id查找元素

document.getElementsByTagName, 通过元素Name查找

document.getElementsByClassName, 通过ClassName查找元素

```
const x = document.querySelectorAll("p.intro");
```

通过CSS选择器查找元素 (id、类名、类型、属性、属性值等) 可选,

```
const x = document.forms["frm1"];
```

```
let text = "";
```

```
for (let i = 0; i < x.length; i++) {
```

```
text += x.elements[i].value + "<br>";
```

```
}
```

```
document.getElementById("demo").innerHTML = text;
```

通过查找forms, id为frm1, 并循环显示所有元素值

更改HTML元素:

element.innerHTML = new content, 修改元素内容

element.attribute = new value, 修改元素的属性的值

element.style.property = new style, 修改元素的样式的属性的值

element.setAttribute (attribute, value), 为元素添加属性与值

添加或删除元素:

document.createElement (element), 创建新元素

document.removeChild (element), 移除元素

document.appendChild (element), 添加一个元素

document.replaceChild (new, old), 使用新元素替换旧元素

document.write (text) 在此位置写入内容, 慎用

添加事件处理程序:

document.getElementById (id) .onclick = function () {code}, 点击执行程序

对HTML元素发生事件之后, 会执行相应的代码:

- 点击了某个元素
- 页面已加载
- 输入字段已更改

查找HTML对象:

document.body, 返回body这个元素

以上类推cookie、doctype、baseURL、documentElement、documentMode、documentURL、embeds、forms、head、images、等返回元素以及内容

004.表单:

表单验证:可以通过JS完成, 下例

```
function validateForm() {  
    let x = document.forms["myForm"]["fname"].value;  
    if (x == "") {  
        alert("Name must be filled out");  
        return false;  
    }  
}
```

如果该表单字段fname为空, 则发出警告并返回false, 阻止表单提交

验证数字输入:

自动的HTML表单验证: 表单验证可以由浏览器自动执行

约束验证: 设置后会限制输入内容, 来保障表单正确提交

约束验证的输入属性:

disabled、max、min、pattern、required、type

约束验证的CSS伪选择器:

: disabled、: invalid、: optional、: required、: valid

005.事件:

对事件发生作出反应:

事件示例:

- 当用户点击鼠标时
- 当网页加载完成后
- 当图像已加载时
- 当鼠标移到元素上时
- 当输入字段改变时
- 提交 HTML 表单时

- 当用户按下按键时

事件属性：将执行的事件分配给元素，事件指代码

`<button onclick="displayDate()">Try it</button>`，元素分配事件

`<script>`

`document.getElementById("myBtn").onclick = displayDate;`

`</script>`事件分配给元素

onload和onunload：

用户进入或离开页面时候会触发事件、一般用于检查访问者浏览器类型和版本、处理cookie

oninput：

通常与用户输入数据时执行的操作有关：下例使用oninput改变输入字段内容

`<input type="text" id="fname" oninput="upperCase()">`

onchange：

通常与输入字段的验证结合使用：下例当用户改变内容时调用函数

`<input type="text" id="fname" onchange="upperCase()">`

onmouseover和onmouseout：

鼠标悬停、移出某个HTML元素时，出发某个功能

onmousedown、onmouseup和onclick

鼠标摁下、鼠标点击完成、鼠标点击会触发事件

006.事件监听器：可以添加到任何DOM对象，不仅是HTML元素

addEventListener () 与removeEventListener ()：

方法将事件处理程序附加到指定元素、而不会覆盖现有的事件处理程序、

可以向同一个元素添加多个事件处理程序、类型任意

语法：`element.addEventListener(event, function, useCapture);`

第一个参数是事件类型、click、mousedown等，不要使用

"on"前缀

第二个参数指调用的函数

第三个参数是布尔值、用于指定是否使用**事件冒泡或者捕获true是捕捉、false是冒泡传播**

向元素添加事件处理程序：

```
element.addEventListener("click", function(){ alert("Hello World!"); });
```

为元素添加事件以及执行函数，函数可以为外部函数

向同一元素添加多个事件处理程序：

```
element.addEventListener("mouseover", myFunction);
```

```
element.addEventListener("click", mySecondFunction);
```

```
element.addEventListener("mouseout", myThirdFunction);
```

向窗口对象添加事件处理程序：

如下，当窗口大小被调整时触发事件

```
window.addEventListener("resize", function(){  
document.getElementById("demo").innerHTML =  
sometext  
;  
});
```

传递参数：

传递参数值、使用匿名函数、可以调整参数调用不同的函数

```
element.addEventListener("click", function(){ myFunction(p1, p2); });
```

事件冒泡与时间捕捉：

DOM中事件传播有两种方式：冒泡与捕捉

事件传播：

指定义事件发生时元素的顺序

冒泡：

最内层元素的事件首先被处理、然后是外层元素的事件

捕捉：

优先处理最外层事件，然后处理内层元素的事件

007.元素节点：

使用DOM创建新元素，必须先创建该元素的节点，然后再将其附加到现有元素

理解为：使用DOM创建元素，两个节点（内容与新元素、新元素与旧元素），两个附加（内容附加新元素、新元素附加旧元素）

创建新元素

创建元素的内容节点

将内容节点附加给元素

找到现有元素

将新建的元素附加给现有元素

appendChild () 方法用将新元素附加给父元素的最后一个子元素

```
<div id="div1">  
<p id="p1">This is a paragraph.</p>  
<p id="p2">This is another paragraph.</p>  
</div>  
<script>  
const para = document.createElement("p"); //创建新元素p  
const node = document.createTextNode("This is new."); //创建文本节点  
para.appendChild(node); //将文本节点附加给新元素  
必须将新元素添加到现有元素  
const element = document.getElementById("div1"); //查找到现有父元素  
element.appendChild(para); //将新元素附加到父元素的最后一个节点  
</script>
```

insertBefore ()：将文本节点添加到父元素的指定子元素之前

```
const child = document.getElementById("p1"); //查找需要插入的位置元素  
element.insertBefore(para, child); //将新元素附加给父元素的指定位置前
```

remove ()：删除现有元素

```
<div>  
<p id="p1">This is a paragraph.</p>  
<p id="p2">This is another paragraph.</p>  
</div>  
<script>
```

```
const elmnt = document.getElementById("p1");//找到需要删除的元素
elmnt.remove();//对该元素执行删除方法
</script>
```

removeChild ()：对于旧版本浏览器，需要找到父节点才可以删除元素

```
<script>
const parent = document.getElementById("div1");
const child = document.getElementById("p1");
parent.removeChild(child);//对父元素使用删除指定子元素的方法
也可以使用如下parentNode方法来定位到该子元素的父元素
const child = document.getElementById("p1");
child.parentNode.removeChild(child);
</script>
```

replaceChild ()：替换父节点的指定子元素

```
<div id="div1">
<p id="p1">This is a paragraph.</p>
<p id="p2">This is another paragraph.</p>
</div>
<script>
const para = document.createElement("p");
const node = document.createTextNode("This is new.");
para.appendChild(node);//将文本节点附加给新元素
const parent = document.getElementById("div1");
const child = document.getElementById("p1");
parent.replaceChild(para, child);//使用新元素替换p1元素
</script>
```

008.DOM集合：

getElementsByTagName () 方法，会返回一个HTMLCollection对象，对象是元素的数组状列表，类似于集合。可以使用索引访问定位

```
const myCollection = document.getElementsByTagName("p");  
myCollection 【1】
```

Collection长度：

```
myCollection.length
```

循环遍历该集合：使用for循环、下例更改每一个颜色

```
const myCollection = document.getElementsByTagName("p");  
for (let i = 0; i < myCollection.length; i++) {  
  myCollection[i].style.color = "red";  
}
```

HTMLCollection不是数组！可以使用循环遍历、数字索引，但是不能使用数组方法，增减元素等

009.DOM节点列表：

部分浏览器使用getElementsByTagName () 返回的对象是NodeList，

大多数浏览器使用querySelectorAll ()，会返回一个NodeList，

所有浏览器都会为childNodes属性返回一个NodeList对象。

NodeList几乎与Collection相同，也可以使用索引访问定位、循环遍历

```
const myNodeList = document.querySelectorAll("p");  
myNodeList 【1】
```

NodeList的长度：

```
NodeList.length
```

循环遍历该集合：下例更改每一个颜色

```
const myNodeList = document.querySelectorAll("p");  
for (let i = 0; i < myNodeList.length; i++) {  
  myNodeList[i].style.color = "red";  
}
```

NodeList与Collection的区别

Collection是文档元素的集合

Nodelist是文档节点的集合，包括元素节点、属性节点、文本节点

Collection可以使用名称、ID、索引来访问

Nodelist只能通过索引访问

Collection始终是实时集合，如果将li元素添加到列表中，collection列表也会发生改变

Nodelist通常是静态集合，添加新元素，列表不会变

getElementsByClassName () 和getElementsByTagName () 会返回一个实时的列表

querySelectorAll () 返回一个静态的列表，

childNodes属性返回一个动态的列表